

全栈开发工程师：HR 动态表单与通用接口引擎

限定条件： 允许使用 TypeScript/Node.js、Python、Go、PHP、Rust 等任意主流语言/框架，**严禁使用 Java**。前端技术栈建议使用 React、Vue 3 或 Svelte。这里所有技术栈都是建议，并不是范围限制（但是依然禁用 Java），您可以尽情发挥。

提交要求： 请将最终成品部署至云端。为方便您进行部署测试，请将您的 SSH 公钥提供给对接人员，我们将为您统一开通 AWS 云服务器实例并提供访问权限。最后请将 repo 地址和在线系统访问地址一并提交。

公司 HR 系统需要支持不同部门、不同岗位的个性化入职信息收集（如技术岗需填 GitHub 链接，销售岗需填驾驶证信息）。HR 团队希望**无需开发人员介入**，即可通过后台配置出全新的入职表单，并自动生成对应的 API 接口进行数据的提交、查询与校验。

场景一：动态表单 Schema 设计与前端渲染引擎

背景： HR 需要定义一个“技术岗位入职表单”，用于记录存档入职人员信息，字段类型包含文本输入、下拉选择、文件上传以及嵌套数组（如“工作履历”）。

Q0：Schema 数据结构设计

请设计一套 JSON Schema，用于描述表单的字段结构，需包含：

- 字段标识 (`key`)、标签 (`label`)、组件类型 (`type` : `input` , `select` , `upload` , `repeater`)。
- 基础校验规则（必填、正则、最大/最小值、`regex`、`email`）。

同时支持：

- 嵌套 Object
- 嵌套 Array
- 字段默认值
- Schema Version
- 字段废弃但历史数据仍存在的情况

Q1：前端动态渲染引擎

写出一个前端组件，接收上述 Schema，自动渲染对应的表单控件并管理表单状态 (`formState`)。

```
1  // Schema 结构参考示例（需由你在代码中进一步完善）
2  {
3    "formId": "tech_onboarding",
4    "title": "技术岗入职信息表",
5    "fields": [ ... ]
```

重点支持：

- 动态添加/删除数据项
- Rule Engine
- 字段及错误提示
- 表单级错误提示
- Dirty State
- Loading State
- Server Validation Error

进阶要求：考虑以下情况并防止 Rule Engine 无限循环：

```
1  A 修改 B
2  B 修改 C
3  C 又修改 A
```

Q2: Schema 生命周期与版本管理

某个表单当前 Schema Version 为 v3，已有 10,000 条历史提交记录。HR 修改了表单：employeeName 改名为 fullName；删除 nickname；新增必填字段 department。请设计 Schema 的生命周期，要求：

- 已发布 Schema 不允许直接修改
- 修改后生成新的 Schema Version
- 历史 Submission 必须保留原 Schema Version
- 新 Submission 使用最新 Published Version

场景二：可配置通用数据 API 引擎

背景：系统不能为每一个新创建的表单单独编写路由代码，必须提供一套通用的 CRUD 接口引擎。

Q0: 通用提交接口

```
POST /api/v1/dynamic-data/:formId
```

- 接口接收 `:formId` 及 payload 数据。
- 自动解析 **Schema**，校验提交的 JSON 数据是否合法。若校验失败返回详细的错误提示（如：字段 email 格式不正确）。
- 校验通过后，将数据持久化存储（数据库设计不限：如 PostgreSQL JSONB、MongoDB 或 SQLite JSON 等）。

Q1: 通用查询与筛选接口

GET /api/v1/dynamic-data/:formId

- 支持通过 JSON 字段进行分页与动态筛选（如：查询所有 `experience > 3` 的员工）。

Q2：字段联动与高级表达式引擎

HR 提出需求：表单字段之间存在联动逻辑。例如：

- 当“是否应届生”选择 `是` 时，隐藏“前公司名称”字段，并取消其“必填”校验。
- 当“期望薪资”大于 `30000` 时，自动显示“需要 HRBP 特别审批”提示文本。

Q3：文件上传安全

动态表单需要支持身份证、合同、简历、银行卡资料等敏感文件上传。请设计文件上传流程。禁止直接将大文件 Binary 保存到 Submission JSON。

Q4：生命周期钩子与扩展机制

为了满足 HR 系统未来的扩展需求（如：提交表单后触发钉钉通知、自动生成 PDF 合同、将候选人同步至第三方考勤系统），通用接口必须支持插件/钩子机制。

Q5：并发修改

两个 HR 同时打开 Form Builder，两人读取 `schemaVersion = 10`。HR A 修改并保存，生成 `version = 11`；与此同时 HR B 也提交自己的修改。请设计解决方案，避免 B 覆盖 A 的修改。可以考虑 Optimistic Locking、ETag、version field。

Q6：审计日志

系统涉及 HR 敏感数据，需要知道谁在什么时候修改了什么。例如 `salary: 30000 -> 35000`。